HILL CLIMBING N-QUEENS

Code:

```
def cost(state):
    conflicts = 0
    n = len(state)

    for i in range(n):
        for j in range(i + 1, n):
            if (state[i] == state[j] or
                abs(state[i] - state[j]) == abs(i - j)):
                conflicts += 1

    return conflicts


def print_board(state):
    n = len(state)

    for row in range(n - 1, -1, -1):
        for col in range(n):
            if state[col] == row:
                print(" Q ", end="")
            else:
                print(" . ", end="")
        print()
    print()


def hill_climbing():
    state = [3, 1, 2, 0]
    n = len(state)
    step = 1

    print("INITIAL BOARD:")
    print_board(state)
    print("Initial State:", state)
    print("Initial Cost:", cost(state))

    while cost(state) != 0:
        current_cost = cost(state)
        neighbors = []

        # Generate neighbors by swapping row positions
        for i in range(n):
            for j in range(i + 1, n):
                neighbor = state.copy()
                neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
                neighbors.append((cost(neighbor), i, j, neighbor))
```

```
        # Select minimum-cost neighbor; break ties by indices
        neighbors.sort(key=lambda x: (x[0], x[1], x[2]))

        best_cost, i, j, best_state = neighbors[0]

        print("\nIteration:", step)
        print("Current State:", state)
        print("Current Cost:", current_cost)

        for neighbor_cost, r1, r2, neighbor in neighbors:
            print("Swap rows:", r1, "and", r2,
                  "| State:", neighbor,
                  "| Cost:", neighbor_cost)

        print("Selected Swap:", i, "and", j)
        print("Selected State:", best_state)
        print("Selected Cost:", best_cost)

        if best_cost >= current_cost:
            print("\nNo better neighbor found. Stopping.")
            print("Final Cost:", current_cost)
            print("Final State:", state)
            print("FINAL BOARD:")
            print_board(state)
            return

        state = best_state
        step += 1

    print("\nGOAL STATE REACHED!")
    print("Final State:", state)
    print("Final Cost:", cost(state))
    print("FINAL BOARD:")
    print_board(state)


hill_climbing()
```

output:

```
INITIAL BOARD:
 Q  .  .  .
 .  .  Q  .
 .  Q  .  .
 .  .  .  Q

Initial State: [3, 1, 2, 0]
Initial Cost: 2

Iteration: 1
Current State: [3, 1, 2, 0]
Current Cost: 2
Swap rows: 0 and 1 | State: [1, 3, 2, 0] | Cost: 1
Swap rows: 0 and 2 | State: [2, 1, 3, 0] | Cost: 1
Swap rows: 1 and 3 | State: [3, 0, 2, 1] | Cost: 1
Swap rows: 2 and 3 | State: [3, 1, 0, 2] | Cost: 1
Swap rows: 0 and 3 | State: [0, 1, 2, 3] | Cost: 6
Swap rows: 1 and 2 | State: [3, 2, 1, 0] | Cost: 6
Selected Swap: 0 and 1
Selected State: [1, 3, 2, 0]
Selected Cost: 1

Iteration: 2
Current State: [1, 3, 2, 0]
Current Cost: 1
Swap rows: 2 and 3 | State: [1, 3, 0, 2] | Cost: 0
Swap rows: 0 and 1 | State: [3, 1, 2, 0] | Cost: 2
Swap rows: 0 and 2 | State: [2, 3, 1, 0] | Cost: 2
Swap rows: 1 and 3 | State: [1, 0, 2, 3] | Cost: 2
Swap rows: 0 and 3 | State: [0, 3, 2, 1] | Cost: 4
Swap rows: 1 and 2 | State: [1, 2, 3, 0] | Cost: 4
Selected Swap: 2 and 3
Selected State: [1, 3, 0, 2]
Selected Cost: 0

GOAL STATE REACHED!
Final State: [1, 3, 0, 2]
Final Cost: 0

FINAL BOARD:
 .  Q  .  .
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .
```

SIMULATED ANNEALING

CODE:

```
import random
import math
```

```
def calculate_conflicts(state):
    """
    Computes the number of conflicting queen pairs.
    Energy/Cost = 0 means a valid solution is found.
    """
    conflicts = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):

            if state[i] == state[j] or abs(state[i] - state[j]) ==
abs(i - j):
                conflicts += 1
    return conflicts

def get_random_neighbor(state):
    """
    Generates a neighbor state by randomly moving one queen
    to a different row in its column.
    """
    neighbor = list(state)
    col = random.randint(0, 7)
    current_row = state[col]


    new_row = random.choice([r for r in range(8) if r != current_row])
    neighbor[col] = new_row
    return neighbor

def simulated_annealing(init_temp=100.0, cooling_rate=0.995,
max_iterations=10000):
    """
    Solves the 8-Queens problem using Simulated Annealing.
    """

    current_state = [random.randint(0, 7) for _ in range(8)]
    current_cost = calculate_conflicts(current_state)

    temp = init_temp

    for i in range(max_iterations):

        if current_cost == 0:
            return current_state, i


        if temp <= 0.0001:
```

```
            break

        neighbor = get_random_neighbor(current_state)
        neighbor_cost = calculate_conflicts(neighbor)

        delta_energy = neighbor_cost - current_cost

        if delta_energy < 0:
            current_state = neighbor
            current_cost = neighbor_cost
        else:
            probability = math.exp(-delta_energy / temp)
            if random.random() < probability:
                current_state = neighbor
                current_cost = neighbor_cost

        temp *= cooling_rate

    return current_state, max_iterations

def print_board(state):
    """Visualizes the board state in a readable text format."""
    for row in range(8):
        line = ""
        for col in range(8):
            if state[col] == row:
                line += " Q "
            else:
                line += " . "
        print(line)

if __name__ == "__main__":
    print("Running Simulated Annealing for 8-Queens...")

    attempts = 0
    while True:
        attempts += 1
        solution, iterations = simulated_annealing()
        cost = calculate_conflicts(solution)

        if cost == 0:
```

```
            print(f"\nSolution found on attempt {attempts} (after
{iterations} iterations):")
            print(f"State Array (Columns 0-7 rows): {solution}\n")
            print_board(solution)
            break
        else:
            continue
```

OUTPUT:

```
Running Simulated Annealing for 8-Queens...

Solution found on attempt 1 (after 1496 iterations):
State Array (Columns 0-7 rows): [2, 5, 1, 4, 7, 0, 6, 3]

. . . . . Q . .
. . Q . . . . .
Q . . . . . . .
. . . . . . . Q
. . . Q . . . .
. Q . . . . . .
. . . . . . Q .
. . . . Q . . .
```